

On-line and Hierarchical Scheduling

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

2025/26

Paulo Baltarejo Sousa

`pbs@isep.ipp.pt`

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Gratitude

- Many thanks to Prof. Luís Miguel Pinho:
 - For providing course material
 - For helping to prepare this course
 - ...

1 On-line Scheduling

2 Hierarchical Scheduling

- Polling Server
- Deferrable Server
- Sporadic Server

1 On-line Scheduling

2 Hierarchical Scheduling

- Polling Server
- Deferrable Server
- Sporadic Server

An example scenario

- The control system of an airplane has the following tasks:
 - A **periodic task** responsible for controlling the pitch, roll and yaw of the airplane, which **has to execute every 60 ms**, it takes **20 ms** to execute and **must output the results in 40 ms**;
 - A **sporadic alarm task**, that when occurring must be handled in 20 ms, taking 5 ms to execute;
 - A periodic logger task, which writes the log of events to the black-box, with a period of 100 ms and an execution time of 50 ms.

	C (ms)	T (ms)	D (ms)
Control	20	60	40
Alarm	5	70	20
Logger	50	100	100

Scheduling approaches

- **Fixed-task priority** – if priorities are assigned per-task before run-time and applied to all jobs of each task and do not change during run-time;
 - Rate Monotonic and Deadline Monotonic (Previous classes)
- **Fixed-job priority** – if the priority of a task might change during run-time but each job has a fixed priority;
- **Dynamic-job priority** – if the priority of a job might change during run-time;
- Complex approaches, integrating some of the above.
 - Hierarchical Scheduling
 - ...

Dynamic priorities (I)

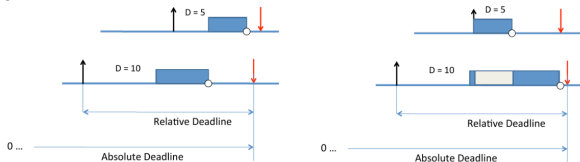
- Order of execution of tasks **is determined during system execution**
 - Based on properties calculated on-line, which specify a “dynamic priority”
 - The ready queue **is updated every time "schedule" properties change**
- Advantages
 - **Easy to scale to many tasks**
 - **Easy to accommodate tasks with dissimilar periods (and sporadics)**
- Disadvantages
 - **Higher runtime complexity and overhead**
 - **Unknown behavior under overloads**
 - in this particular, it is not possible **to know beforehand which tasks are delayed**

Dynamic priorities (II)

- Fixed-job priority
 - **First Come First Serve (FCFS)**
 - Next to execute is the task with a larger waiting time
 - **Earliest Deadline First (EDF):**
 - **Assign priority to jobs based on absolute deadline:** earlier the deadline, higher the priority.
- Dynamic-job priority
 - **Least Laxity First (LLF) / Least Slack First (LSF)**
 - Next to execute is the task with the smaller spare time

Earliest Deadline First (EDF)

- Fully on-line algorithm
 - The runnable tasks **are executed in the order determined by their absolute deadlines**
 - The next task to run is always the one with the shortest (nearest) deadline



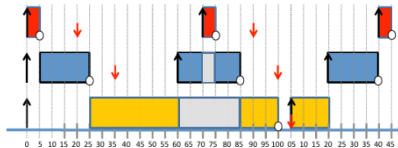
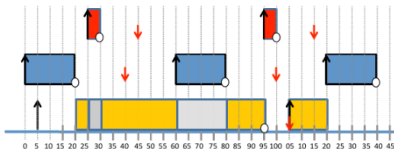
- Although it is usual to know the relative deadlines of each process (e.g. 25 ms after release), **the absolute deadlines are computed at run time** and hence the scheme is described as dynamic
- It allows better utilization in the system, **although in overloads there is an unpredictably issue**
 - No way to know which task gets a deadline miss – in FPS is always the lowest priority one

EDF: critical instant (I)

- There is no well defined critical instant for EDF.

	C (ms)	T (ms)	D (ms)	Priority	Arrival instant
Control	20	60	40	-	0
Alarm	5	70	20	-	25
Logger	50	100	100	-	5

	C (ms)	T (ms)	D (ms)	Priority	Arrival instant
Control	20	60	40	-	0
Alarm	5	70	20	-	0
Logger	50	100	100	-	0



EDF: critical instant (II)

- The **worst-case response time for each task does not occur when all tasks are released at a critical instant:**
 - It is necessary to consider all task releases to see which one suffers the maximum interference.
- **The full task set will repeat its execution every hyper-period:**
 - The least common multiple (LCM) of the tasks' periods.
 - Example, assume 4 tasks with periods 24, 50, 73 and 101 :
 - The LCM is 4 423 800;
 - To find the worst-case response time for EDF requires each release within 4 423 800 time units to be considered;
 - With FPS only the first release of each task needs to be analysed (in this case, 101 time units).

EDF: Utilization based tests (I)

- For pre-emptive FPS ($D = T$)
 - if $U > 1$, the system is never schedulable, since CPU utilization is higher than 100%
 - If $U \leq n(2^{\frac{1}{n}} - 1)$, the system is schedulable
- For pre-emptive EDF ($D = T$)
 - if $U > 1$, the system is never schedulable, since CPU utilization is higher than 100%
 - If $U \leq 1$, the system is always schedulable
- For pre-emptive EDF ($D < T$)
 - If $U^* > 1$, the system is never schedulable (U^* is density $\Rightarrow \frac{C_i}{D_i}$)
 - If $U^* \leq 1$, the system is always schedulable
- For pre-emptive EDF ($D > T$)
 - Use $U_i = \frac{C_i}{\min\{D_i, T_i\}}$

EDF: Utilization based tests (II)

- Remember pre-emptive FPS ($D = T$)

	C (ms)	T=D(ms)	U(Ci/Ti)
τ_1	2	10	0,20
τ_2	9	15	0,60
τ_3	1	25	0,04
$U (\sum C_i / T_i) =$			84%

$$3(2^{1/3} - 1) \approx 78\%$$

$$U = \sum_{i=1}^n \frac{C_i}{T_i} = 84\% > 78\%$$

- With EDF this taskset is guaranteed to be schedulable

EDF: Analysis

- Non-preemptive case
 - With dynamic priorities, the non-preemptive case becomes more complex, as the concept of blocking is also dynamic
 - A blocking task is any task that may have a nearer absolute deadline
 - It is necessary to consider the processor load within a synchronous busy period
 - The longest period of continuous CPU load
 - But it is then necessary to check all cases within such period
- What about response time analysis?
 - EDF response time analysis is more complex than FPS since it is not known what job suffers the maximal interference (there is no concept such as the critical instant)
 - It is known that the worst-case response time occurs within the synchronous busy period
 - Same issue as for non-preemptive case
- This is out of scope of the course

EDF: features

- **EDF is considered to be fair**
 - In the sense **that all tasks see their priority raised as they are delayed close to their deadline**, independently of their period or other static parameter
 - There are no "more important" tasks
 - This also means that in an overload it is not possible to know which tasks will fail deadlines
- EDF also avoids preemptions when tasks come closer to their deadlines
 - Means that faster appearing tasks (smaller periods) are heavily influenced by current executing situation leading (or could lead) to higher jitter

EDF: Exercises (I)

- Trace the execution of the following taskset, with FPS and EDF, for 100 ms

	C (ms)	T=D (ms)	Arrival
τ_1	10	30	0
τ_2	10	40	0
τ_3	21	60	0

- Check if the following taskset is schedulable under FCFS (First Come First Serve)

	C (ms)	T=D (ms)
τ_1	25	50
τ_2	10	60
τ_3	20	80

EDF: Exercises (II)

- 3 EDF, preemptive, what is the minimum period for task τ_1 , so that the following taskset is schedulable

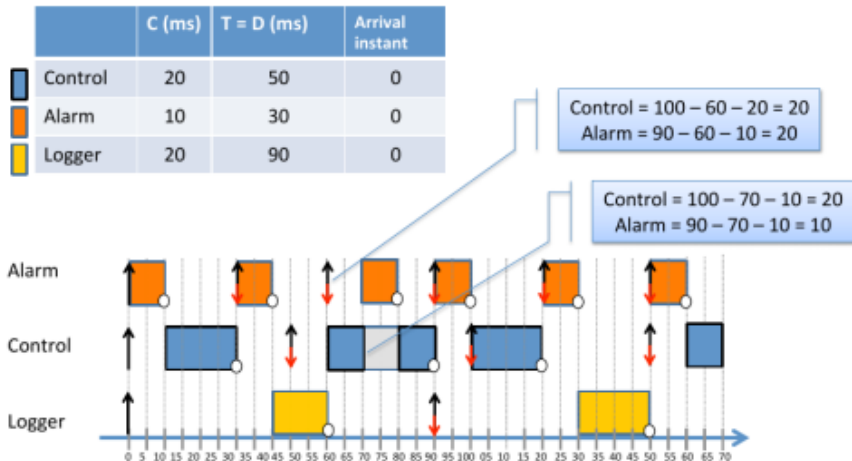
	C (ms)	T=D (ms)
τ_1	25	?
τ_2	10	60
τ_3	20	80

Least Laxity First (LLF) (I)

- The Laxity, $L_i(t)$, of a task at time t is computed as $L_i(t) = d_i^{rem}(t) - c_i^{rem}(t)$ and **provides a measure of urgency for this task**;
 - During execution, in addition to its C_i , T_i , and D_i parameters, a task τ_i is also characterized by its remaining execution time, $c_i^{rem}(t)$, and by its remaining time to absolute deadline, $d_i^{rem}(t)$, at time t ;
- Whenever the $L_i(t)$ is zero the task must be scheduled to execute immediately, otherwise it will miss its deadline.
 - If two or more tasks have the same laxity, a laxity-tie occurs.
 - Once a laxity-tie occurs, context switches take place every scheduling point until the tie breaks

LLF (II)

- It is also known as Least Slack Time First.
- Critical instant = EDF.



1 On-line Scheduling

2 Hierarchical Scheduling

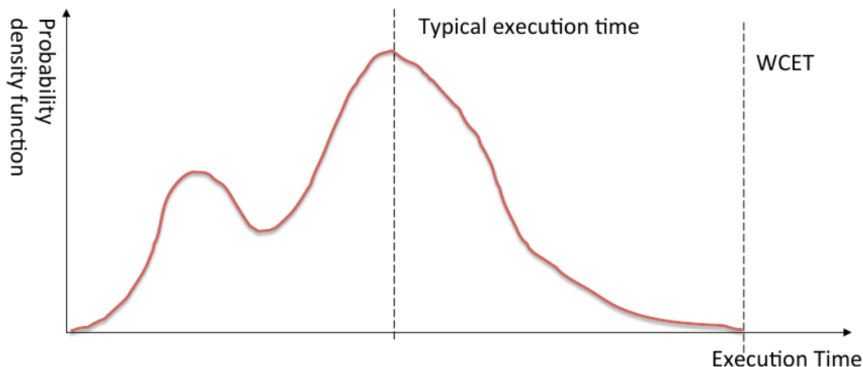
- Polling Server
- Deferrable Server
- Sporadic Server

What is?

- A hierarchical scheduling system **comprises one or more components in a scheduling hierarchy**.
 - Each component **recursively encapsulates one or more scheduling heuristics**.
 - First, priority levels are assigned, and the ready task with the highest priority is selected
 - If there are more than one task in the same priority level, a second scheduling decision is taken, e.g. Earliest Deadline First, or Round-Robin with the priority
- Several Operating Systems support this model:
 - Linux, Zephyr, and so on.

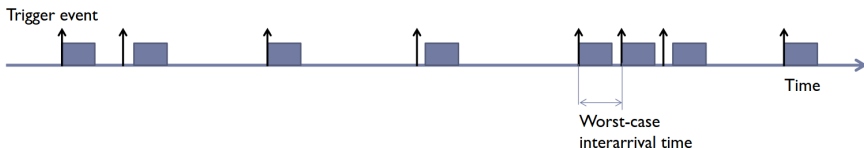
Computing Worst-case execution time

- Worst-case execution time (C_i) **assumptions keep resources unused for most of the time:**
 - Usually, tasks' C_i is rare and **much longer than the average case;**
 - It is increasingly difficult to compute C_i bounds in modern hardware **without introducing excessive pessimism.**



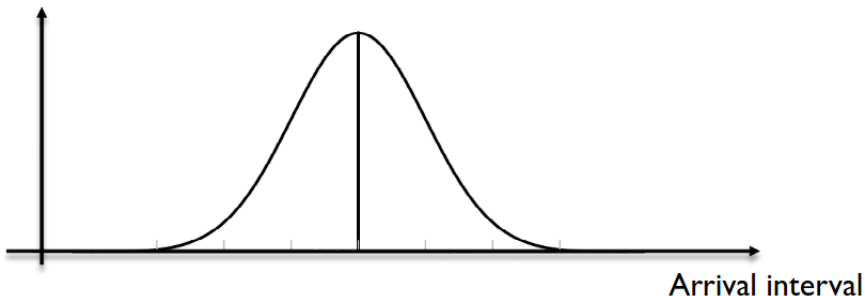
Supporting Sporadic Tasks

- Sporadic activities **have a maximum frequency of repetition**:
 - Available from the specification
 - Worst-case (or the minimal) interarrival time **is highly pessimistic**.



Supporting Aperiodic Tasks

- Aperiodic activities **have no defined periodic information**
 - Potentially only stochastic characterization

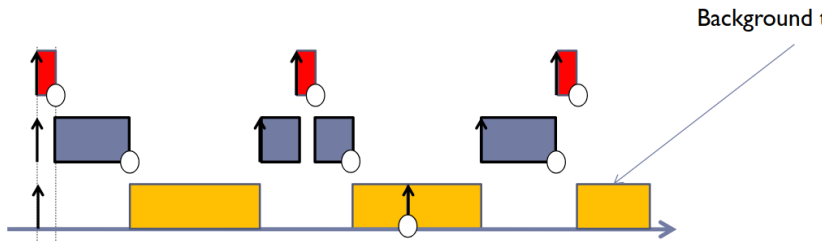


Open real-time systems

- Previous scheduling approaches work with completely defined systems
 - The number of tasks is fixed;
 - Each task is characterised by known parameters (C_i , T_i , and D_i).
- **What about when some of these parameters are not known**
- Using the **worst-case execution time and/or the minimal interarrival time would produce a very pessimistic approach?**
- What about **aperiodic events?**
- Usually systems **combine different types of tasks.**

Supporting Aperiodic Tasks

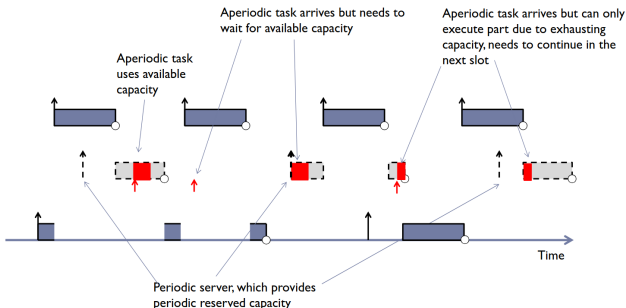
- One approach is to execute **in background**
 - Give them **the lowest priority** or “infinite” deadline
 - Aperiodic tasks only execute “best effort”, if other tasks are not executing



- Easy to implement and does not interfere with periodic tasks
 - Small overhead only of handling the trigger event (interrupts)
- But **can suffer large service delays**
- Poor performance for real-time aperiodic tasks** (e.g., alarms), suitable for non-real-time tasks (e.g. logging)

Using Servers (I)

- An approach to support aperiodic is to **use servers**
 - A special "task" whose purpose is just to **execute active aperiodic tasks**
 - This "task" is called server and **it is characterized by a period T_s and a capacity (or budget) Q_s (actually, processor time or bandwidth).**
 - This way, it is possible to **handle the server as if it was a periodic task with the required priority/deadline.**



Using Servers (II)

- A flexible scheduling approach **to increase resource usage** because **it could enforce temporal isolation**:
 - It allows for predictable performance by encapsulating tasks within a "server" meaning a malicious or malfunctioning task cannot consume more than its allocated processor bandwidth.

Using Servers (III)

- There are various types of servers designed to handle aperiodic tasks, which can operate with either fixed or dynamic priorities.
- These differ in several aspects:
 - Impact on the schedulability of the periodic tasks
 - Average response time of aperiodic tasks
 - Overheads and implementation cost
- Fixed priority servers include:
 - Polling server
 - Deferrable server
 - Sporadic server
- Dynamic priority servers include:
 - Total Bandwidth Server (TBS)
 - Constant Bandwidth Server (CBS)

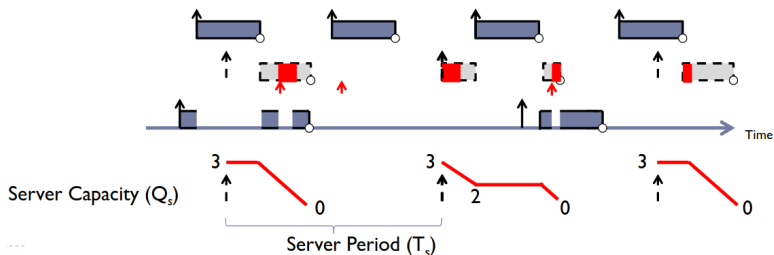
1 On-line Scheduling

2 Hierarchical Scheduling

- Polling Server
- Deferrable Server
- Sporadic Server

Polling Server (I)

- Equivalent to executing a **periodic task with T_s period and Q_s execution time (denoted capacity)**
 - Aperiodic requests are executed during the time assigned by the scheduler
 - **Even if no requests, the server capacity is used**
 - The capacity is replenished every period



Polling Server (II)

- A polling server is easy to implement, with just a single queue for aperiodic requests and monitoring the capacity being used
- The average response time for aperiodic requests is better compared to background execution because these tasks can run at a higher priority level
 - However, there are still periods of unavailability because of the server's period
- Effect on schedulability of the periodic taskset **is similar to that of the corresponding virtual periodic task:**

$$U_p + U_s \leq (n + s)(2^{\frac{1}{n+s}} - 1)$$

- Where n is the number of periodic tasks, s the number of servers, U_p is the total utilization of periodic tasks, U_s is the total servers utilization
- Replacing servers for an equivalent periodic task in the schedulability analysis, allows to use the usual limit (n is the total number of tasks, including servers): $U \leq n(2^{\frac{1}{n}} - 1)$

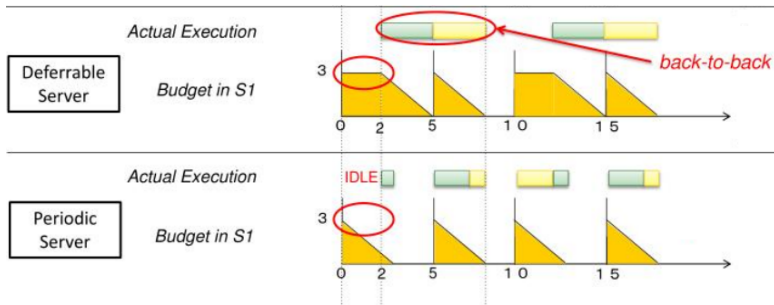
1 On-line Scheduling

2 Hierarchical Scheduling

- Polling Server
- Deferrable Server
- Sporadic Server

Deferrable Server

- It can defer capacity to when it is needed
 - Implementation is similar to polling server
 - Response time to the aperiodic requests is improved in average (compared to polling server) because it is possible to use the capacity during the period, thus reducing unavailability periods
 - Delayed executions increase the load on the future, and it is possible having two consecutive executions (back-to-back execution)



1 On-line Scheduling

2 Hierarchical Scheduling

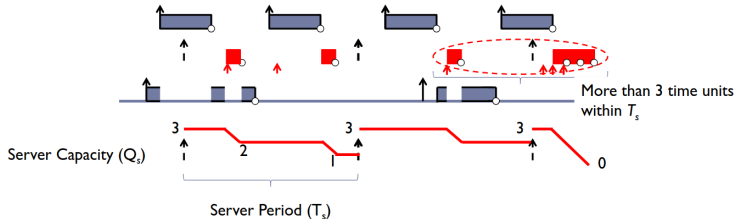
- Polling Server
- Deferrable Server
- Sporadic Server

Sporadic Server (I)

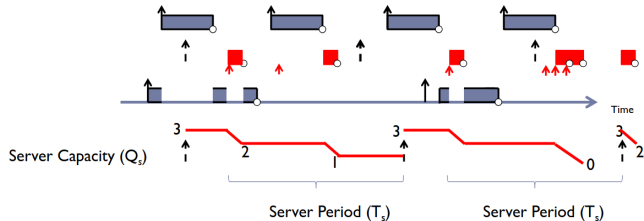
- Bandwidth preserving servers
 - Sporadic servers are designed to improve over a deferrable server.
- The consumption and replenishment rules of sporadic server algorithms ensure that each sporadic server with period T_s and budget Q_s never demands more processor time than the periodic task (T_s, Q_s) in any time interval
- Consequently, we can treat the sporadic server exactly like the periodic task
 - A system of periodic tasks containing **a sporadic server may be schedulable while the same system containing a deferrable server with the same parameters is not.**

Sporadic Server (II)

- Deferrable Server



- Sporadic Server

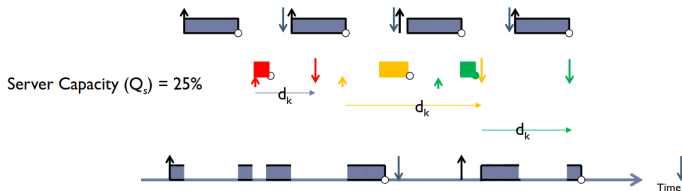


- It also defers replenishment

Total Bandwidth Server (TBS) (I)

- The Total Bandwidth Server (TBS) is a dynamic priority server
 - Purpose is to handle aperiodic requests in EDF as early as possible
 - Maintaining a bandwidth to limit the impact on the taskset
 - When a new request (job) with c_k execution time arrives at instant t_k , a deadline d_k , is assigned to the server with utilization U_s :

$$d_k = \max\{t_k, d_{k-1}\} + \frac{c_k}{U_s}$$
 - Where d_{k-1} is the deadline of the previous job of the server, or zero if none
 - We actually do not need to calculate server period and capacity, just utilization (U_s)

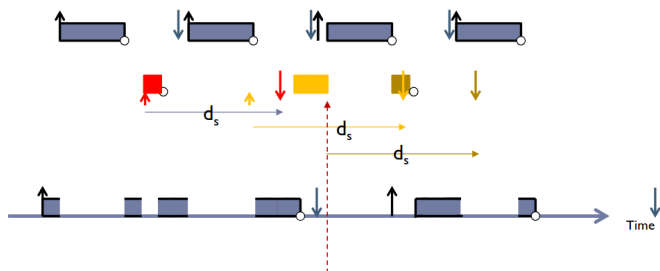


TBS (II)

- Complexity of implementation is relatively small since it is only necessary to calculate the deadline when a request arrives
- The average response time is shorter than with dynamic priorities versions of the fixed-priority servers
- The impact on the schedulability of the taskset is similar to that of a periodic task with the same bandwidth as that of the server:
$$U_p + U_s \leq 1$$
- However
 - TBS requires that we **know each job c_k before execution**
 - TBS is vulnerable to overruns since there is no control over the actual execution time

Constant Bandwidth Server (CBS) (I)

- The Constant Bandwidth Server (CBS) is a dynamic priority server designed to solve the problem of TBS, **enforcing bandwidth isolation**



Job did not finish and capacity is exhausted – deadline is postponed

CBS (II)

- The deadline assignment scheme used by CBS **enforces the server bandwidth independently of the aperiodic load that the server actually executes.**
- If a task executing inside a CBS executes for longer than expected, **its deadline is automatically postponed, which in practice corresponds to lowering the task priority**
- The complexity of implementing a CBS is higher than that of the TBS due to the required capacity management.
- There is still one single ready queue, for periodic and aperiodic tasks, managed by deadlines
- The average response time to aperiodic requests is similar to that of the TBS.
- The impact on the schedulability of the periodic subsystem is equal to that of a task with the same parameters than the server:
$$U_p + U_s \leq 1$$

CBS (III)

- The main motivation to use a CBS resides in the bandwidth isolation that it provides.
- A CBS can be used for:
 - Protecting a system of tasks overruns
 - Guaranteeing a minimal service to aperiodic real-time tasks
 - Reserve bandwidth for any activity
- Linux implements a variant of CBS

CBS (IV)

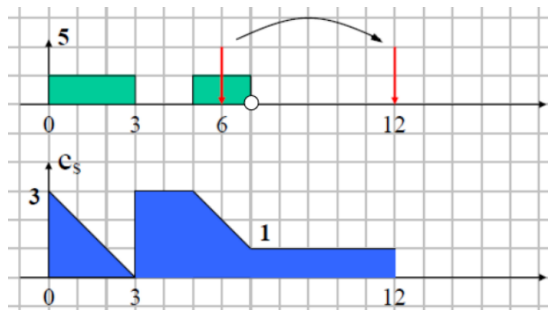
- A CBS is characterised by:
 - a currently available budget c ;
 - a dynamically determined server deadline d ;
 - A pair (Q, T) , where Q is the maximum budget and T is the period of the server.
- The ratio $U_s = Q/T$ is denoted as the server's utilisation;
- Each job served by a CBS **is assigned a dynamic deadline equal to the currently assigned server's deadline**;
- As the job executes, **the budget c is decreased by the same amount**;
- When a job finishes, the next pending job, if any, **is served using the current budget c and deadline d** , otherwise, if there are no pending jobs, the server becomes idle.

CBS (IV)

- Every time $c = 0$, the **server budget is recharged to the maximum value Q and the server deadline is postponed by a period T** ;
 - By postponing the deadline, the task remains eligible for execution;
- As a consequence, overruns occurring on a served task will only delay that task, without compromising the bandwidth assigned to other tasks

CBS (V)

- Server parameters: $Q = 3$, $T = 6$ and $D = T$;
- Aperiodic task. $C = 5$

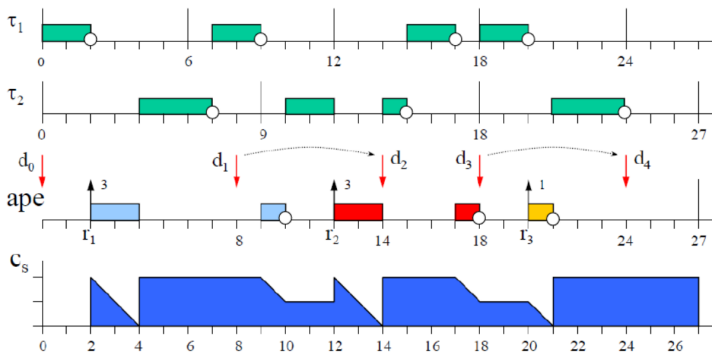


- At time $t = 0$ the server's budget is replenished to 3 and its deadline is $d_s = T_s = 6$.
- At time $t = 3$ the server's budget is exhausted, so its deadline is postponed to $d_s + T_s = 6 + 6 = 12$ and the budget is replenished to 3.

CBS (VI)

- Assume the following task set:
 - Two hard real-time tasks τ_1 and τ_2 ;
 - An aperiodic task τ_3 served by a CBS server with a reserved bandwidth $Q = 2$ and period $T = 6$.

	C (ms)	T = D (ms)
τ_1	2	6
τ_2	3	9



Extensions to CBS

- Bandwidth assignment is critical for the good performance of the system. A careful profiling of the computational requirements of the task is necessary:
 - If the bandwidth is excessive there is an under-utilisation of the system or, what is worst, not enough bandwidth for the rest of the tasks;
 - If the budget is too small the task will not be able to fulfil its requirements.
- An efficient reclaiming and redistribution of unused reserved capacities can significantly reduce the tardiness of soft real-time tasks:
 - Several algorithms: CASH, GRUB, IRIS, CSS, ...